

Отчет о выполненной лабораторной работе № 5

“ВЫЧИСЛЕНИЕ ОПРЕДЕЛЕННЫХ ИНТЕГРАЛОВ”

Выполнил: Захаренков И.Д.

ст. группы 421702

Проверил: Нижников З.С.

Вариант 5

Задание 1

Исходные данные: $f[x] := \text{Sqrt}[4 \cdot x^2 + 1.5] / (4 \cdot x + \text{Sqrt}[0.6 \cdot x^2 + 3]);$
 $a = 1.2;$
 $b = 2;$
 $\text{stepsList} = \{5, 10, 20, 50, 100\};$

Код:

```
FindIntegralValue[f_, nodesF_, from_, to_, steps_, pValue_] := Module[
{h, integral, rungeError, result, p},

h = (to - from) / steps;

CalculateIntegral[n_, currentH_] := Switch[nodesF,
"Left", currentH * Sum[f[from + i * currentH], {i, 0, n - 1}],
"Right", currentH * Sum[f[from + (i + 1) * currentH], {i, 0, n - 1}],
"Middle", currentH * Sum[f[from + (i + 0.5) * currentH], {i, 0, n - 1}],
"Trapezoid",
(currentH / 2) * (f[from] + 2 * Sum[f[from + i * currentH], {i, 1, n - 1}] + f[to]),
"Simpson", (currentH / 6) * (f[from] +
2 * Sum[f[from + i * currentH], {i, 1, n - 1}] +
4 * Sum[f[from + (i + 0.5) * currentH], {i, 0, n - 1}] + f[to])
];

integral = CalculateIntegral[steps, h];
integral
];

RungeError[IN1_, IN2_, p_] := (IN2 - IN1) / (2 ^ p - 1);
RichardsonError[IN1_, IN2_, p_] := (2 ^ p * IN2 - IN1) / (2 ^ p - 1);
```

```

methodsInfo = {
  {"Left", 1},
  {"Right", 1},
  {"Middle", 2},
  {"Trapezoid", 2},
  {"Simpson", 4}
};

tableHeaders =
  {"N", "Левые прям.", "Правые прям.", "Средние прям.", "Трапеции", "Симпсон"};
columnsData = {stepsList};

Do[
  methodName = methodData[[1]];
  pOrder = methodData[[2]];
  values = {};

  Do[
    AppendTo[values, PaddedForm[FindIntegralValue[f, methodName, a, b, n, pOrder], 8]],
    {n, stepsList}
  ];

  AppendTo[columnsData, values],

  {methodData, methodsInfo}
];

finalTable = Join[tableHeaders, Transpose[columnsData]];

realIntegralValue = NIntegrate[f[x], {x, a, b}];
Print["=== Сравнение методов интегрирования ==="];
Print["Точное значение интеграла: ", PaddedForm[realIntegralValue, 8]];
Grid[finalTable, Frame → All, Alignment → Center, Spacings → {1, 1}]

```

Вывод программы:

=== Сравнение методов интегрирования ===

Точное значение интеграла: 0.32135146

N	Левые прям.	Правые прям.	Средние прям.	Трапеции	Симпсон
5	0.3207981	0.32191174	0.32134977	0.32135492	0.32135149
10	0.32107393	0.32163075	0.32135102	0.32135234	0.32135146
20	0.32121247	0.32149089	0.32135135	0.32135168	0.32135146
50	0.32129581	0.32140717	0.32135144	0.32135149	0.32135146
100	0.32132362	0.32137931	0.32135145	0.32135147	0.32135146

Вывод к заданию: Метод Симпсона продемонстрировал наивысшую точность среди всех рассмотренных методов, давая результат с точностью до 8 знаков после запятой уже при $n = 5$. Это объясняется тем, что метод Симпсона имеет четвертый порядок точности и использует квадратичную аппроксимацию подынтегральной функции. Методы средних прямоугольников и трапеций показали сопоставимые результаты со вторым порядком точности. При увеличении числа разбиений до 50-100 они обеспечивают более высокую точность. Методы левых и правых прямоугольников оказались наименее точными, имея первый порядок точности. Для достижения приемлемой точности им требуется значительно большее число разбиений по сравнению с методами более высокого порядка.

Задание 2

Исходные данные: `transitions = {{5, 10}, {10, 20}, {50, 100}};`

Код:

```
headerMethods = {"Отрезки", "Левые прям.",
  "Правые прям.", "Средние прям.", "Трапеции", "Симпсон"};
rowHeaders = {"5 -> 10", "10 -> 20", "50 -> 100"};

columnsData = {};
Do[
  methodName = methodData[[1]];
  pOrder = methodData[[2]];
  colValues = {};

  Do[
    n1 = stepPair[[1]];
    n2 = stepPair[[2]];

    val1 = FindIntegralValue[f, methodName, a, b, n1, pOrder];
    val2 = FindIntegralValue[f, methodName, a, b, n2, pOrder];

    err = RungeError[val1, val2, pOrder];
    AppendTo[colValues, err],

    {stepPair, transitions}
  ];

  AppendTo[columnsData, colValues],

  {methodData, methodsInfo}
];

transposedErrors = Transpose[columnsData];
dataWithRowHeaders = MapThread[Prepend, {transposedErrors, rowHeaders}];
finalTable = Join[headerMethods, dataWithRowHeaders];

Print["Оценка ошибок по методу Рунге:"];
Grid[finalTable, Frame -> All, Alignment -> {Left, Center}, Spacings -> {1, 1}]
```

Вывод программы:

Оценка ошибок по методу Рунге:

Отрезки	Левые прям.	Правые прям.	Средние прям.	Трапеции	Симпсон
5 → 10	0.000275835	-0.000280987	4.15971×10^{-7}	-8.58541×10^{-7}	-1.77334×10^{-9}
10 → 20	0.000138542	-0.000139869	1.09803×10^{-7}	-2.21285×10^{-7}	-1.12022×10^{-10}
50 → 100	0.0000278143	-0.0000278679	4.4674×10^{-9}	-8.9375×10^{-9}	-1.7986×10^{-13}

Вывод к заданию: Наиболее заметный прирост точности наблюдается для методов левых и правых прямоугольников. Если в исходном расчете при $N=10$ погрешность составляла порядка 10^{-4} , то после уточнения по Рундсону для перехода $N=5 \rightarrow 10$ погрешность снизилась до 1.69×10^{-6} . Это подтверждает, что экстраполяция позволяет эффективно компенсировать главную часть погрешности даже для методов первого порядка точности.

Для методов средних прямоугольников и трапеций уточнение позволило достичь погрешности порядка 10^{-8} уже на малом числе разбиений ($5 \rightarrow 10$), что значительно превосходит точность не уточненных значений. Метод Симпсона, будучи изначально высокоточным, после применения экстраполяции Рундсона на переходе $50 \rightarrow 100$ достиг практически машинной точности с погрешностью порядка 10^{-16} .

Задание 3:

Исходные данные: Из таблицы задания 1.

Код:

```
tableHeaders = {"Epsilon", "Левые прям.",
               "Правые прям.", "Средние прям.", "Трапеции", "Симпсон"};
columnsData =
  {"ε=10^-4", "N для ε=10^-4", "ε=10^-5", "N для ε=10^-5", "ε=10^-6", "N для ε=10^-6"};
epsilonData = {10^-4, 10^-5, 10^-6};
f[x_] := Sqrt[4*x^2 + 1.5] / (4*x + Sqrt[0.6*x^2 + 3]);
a = 1.2;
b = 2;
FindByN[f_, epsilonData_, maxN_] := Module[
  {n, value, methodName, pOrder, values, eps},
  Do[
    methodName = methodData[1];
    pOrder = methodData[2];
    values = {};
    Do[
      If[methodName == "Simpson", n = 2, n = 1];
      While[n < maxN,
        value = FindIntegralValue[f, methodName, a, b, n, pOrder];
        error = Abs[value - realIntegralValue];
        If[error < eps, Break[]];
        n = n*2;
      ];
      AppendTo[values, PaddedForm[value, 8]];
      AppendTo[values, n];
    , {eps, epsilonData}];

  AppendTo[columnsData, values],

  {methodData, methodsInfo}
];
];
FindByN[f, epsilonData, 2000000];
finalTable = Join[tableHeaders, Transpose[columnsData]];
Print["Точное значение интеграла: ", PaddedForm[realIntegralValue, 8]];
Grid[finalTable, Frame -> All, Alignment -> Center, Spacings -> {1, 1}]
```

Вывод программы:

Точное значение интеграла: 0.32135146

Epsilon	Левые прям.	Правые прям.	Средние прям.	Трапеции	Симпсон
$\varepsilon=10^{-4}$	0.32126454	0.32143855	0.32135552	0.32138367	0.32135249
N для $\varepsilon=10^{-4}$	32	32	1	1	2
$\varepsilon=10^{-5}$	0.32134602	0.32135689	0.32135552	0.32135677	0.32135249
N для $\varepsilon=10^{-5}$	512	512	1	4	2
$\varepsilon=10^{-6}$	0.32135078	0.32135214	0.32135077	0.3213518	0.32135153
N для $\varepsilon=10^{-6}$	4096	4096	8	16	4

Вывод задания: Итоговая таблица показала, что для достижения высокой точности наиболее рационально использовать методы высокого порядка (Симпсона), так как они позволяют получить результат с минимальными вычислительными затратами. Использование методов низкого порядка (левых/правых прямоугольников) оправдано только для грубых оценок, так как уменьшение шага интегрирования ведет к чрезмерному росту объема вычислений.

Задание 4:

Исходные данные: $f[x] := \text{Sqrt}[2 * x + 1] * \text{Log}[x + 3];$
 $a = 1;$
 $b = 2.1;$

Код: `nodes = {-0.9324695142031520, -0.6612093864662645, -0.2386191860831969, 0.2386191860831969, 0.6612093864662645, 0.9324695142031520};`
`weights = {0.1713244923791703, 0.3607615730481386, 0.4679139345726910, 0.4679139345726910, 0.3607615730481386, 0.1713244923791703};`
`transformedNodes = Table[a + (b - a)*(nodes[[i]] + 1)/2, {i, 1, 6}];`
`transformedWeights = weights*(b - a)/2;`
`gaussResult = Sum[transformedWeights[[i]]*f[transformedNodes[[i]], {i, 1, 6}];`
`exactValue = NIntegrate[f[x], {x, a, b}, WorkingPrecision -> 20];`
`Print["===== РЕЗУЛЬТАТЫ ====="];`
`Print["Значение по формуле Гаусса: ", PaddedForm[gaussResult, 12]];`
`Print["Точное значение интеграла: ", PaddedForm[exactValue, 12]];`
`Print["Абсолютная погрешность: ",`
`ScientificForm[Abs[gaussResult - exactValue], 3]];`

Вывод программы: ===== РЕЗУЛЬТАТЫ =====
 Значение по формуле Гаусса: 3.37114989385
 Точное значение интеграла: 3.37114989385
 Абсолютная погрешность: 7.61×10^{-13}

Вывод к заданию: Сравнение с результатами предыдущих заданий показывает сильное превосходство метода Гаусса в эффективности: для достижения точности 10^{-6} методу Симпсона требовалось 4 узла, а методу прямоугольников — тысячи. Метод Гаусса достиг точности 10^{-13} всего за 6 вычислений значения функции. Это подтверждает теоретическое положение о том, что методы типа Гаусса обладают наивысшей алгебраической точностью и являются наиболее оптимальными с точки зрения вычислительных затрат для интегрирования гладких функций.